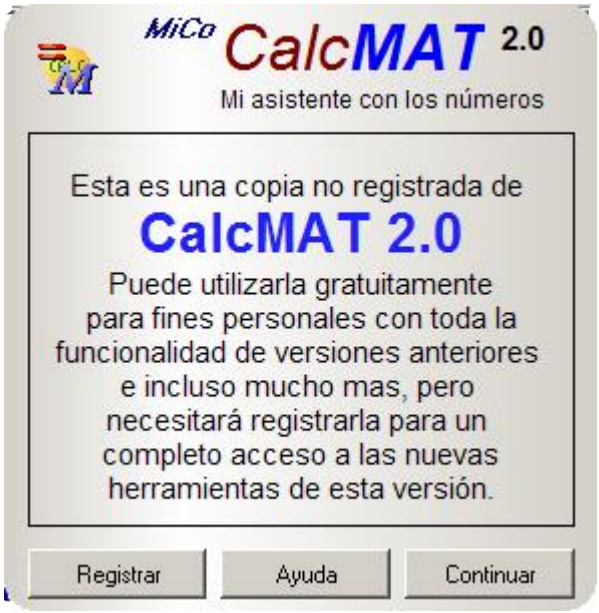
Programa:	CalcMAT 2.0	
Protección:	Serial + Crippleware	
Objetivos:	Liquidar las limitaciones.	
URL descarga:	www.migacon.com	
Dificultad:	Fácil	
Herramientas:	SoftIce + w32dasm + UltraEdit32	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 21-10-04

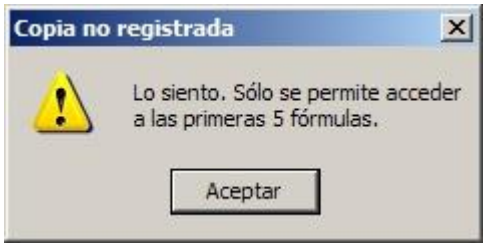
Hola a todos una vez más ;-)

Este es un programa en modo de ***evaluación*** para aplicaciones físicas y matemáticas de cualquier tipo, tiene infinidad de fórmulas y funciones cuyo autor nos permite utilizar sin limitación temporal, pero en su lugar tenemos una ***nag*** que veremos en cada ejecución del programa y también en cada cierre, antes de nada decir que el crack del programa lo he hecho de un modo poco convencional, se asemeja un poco a aquello de ensayo y error aunque no tan descarado tampoco, jeje...

Así pues sin más dilación vamos a instalar el programa y lo ejecutamos para encontrarnos de morros con la siguiente ***nag*** que ya habíamos citado anteriormente:



Como se puede observar ésta nos invita a registrarnos o a evaluar el programa, para esto último deberemos pulsar sobre el botón '***Continuar'*** y entraremos en el auténtico programa, si lo hacemos y investigamos un poco veremos que algunas funciones están deshabilitadas y si intentamos utilizarlas recibiremos un mensaje donde el autor nos indica que nos estarán disponibles en la versión de evaluación, en realidad estas funciones son las añadidas desde la última versión a la actual y como digo ***no las podremos utilizar mientras no estemos registrados***. Bueno ya sabemos algo más, el mensaje es el siguiente aunque hay distintos según que función hayamos intentado utilizar:



En este caso concreto es si queremos usar una ***fórmula*** que está a partir de la ***quinta*** posición (no incluida) dentro del campo '***formulario ->Nombre de fórmula***' aunque como digo hay bastantes más restricciones, que yo haya visto y/o contado son las siguientes:

- ***Formulario / Nombre de fórmula***: No podemos acceder a las fórmulas que ocupan las posiciones superiores a la quinta, aunque si a éstas.

- ***Formulario / Guardar cambios***: No está permitido guardar los cambios realizados.

- ***Conversor***: No se puede acceder a las magnitudes situadas a partir de la quinta posición.

- ***Conversor(2)***: Aun dentro de las magnitudes situadas en las cinco primeras posiciones no podremos utilizar las Unidades que sobrepasan la quinta posición.

- ***Estadísticas***: No se permite evaluar más de 20 datos simultáneamente.

Bueno queda claro que el uso es bastante limitado en cuanto a "***movimientos***" además decir que tras el mensaje de error nos mostrará también la nag que vemos al inicio y al final del programa recordándonos que nos registremos, toda una putada vaya **:(**

Visto lo visto vamos a tomar cartas en el asunto y a intentar resolver tantas limitaciones que nos impiden evaluar el programa tranquilamente, tras ello el que quiera que lo registre y quien no es su problema, cada uno sabe lo que hace, aunque en este tuto no nos vamos a registrar pues simplemente le vamos a eliminar las ventanitas nag y diferentes limitaciones que pueda tener, no vamos a tratar de sacar un serial válido, entre otras cosas porque le he estado echando un vistazo y no veo nada claro.

Al Ataque!!

Desensamblamos el ejecutable para mirar entre sus cadenas para ver que tenemos, lo podemos hacer con ***w32dasm*** o con ***IDA Pro***, yo utilizaré w32dasm, lo hacemos pero no vemos nada interesante así que procederemos con ***SoftIce***, lo cargamos en el ***Symbol Loader*** y lo lanzamos, ***SICE*** saltará justo en la primera instrucción de nuestro ejecutable, en este momento vamos a ***trazar_desde_el_inicio***, iremos trazando con ***F10*** sin entrar de momento en los ***calls*** para que no se nos haga eterno el ***traceado***, en fin que vamos bajando hasta encontrarnos con esto:

```
:0044E4BF 8B45FC              mov eax, dword ptr [ebp-04]
:0044E4C2 8B4024              mov eax, dword ptr [eax+24]
:0044E4C5 50                  push eax

* Reference To: user32.MessageBoxA, Ord:0000h
                |
:0044E4C6 E8D98BFBFF          Call 004070A4              ;aplicación desactualizada!!
:0044E4CB 8945F8              mov dword ptr [ebp-08], eax
:0044E4CE 33C0                xor eax, eax
:0044E4D0 5A                  pop edx
```

Aquí tenemos el primer mensajito que podemos eliminar tranquilamente para que no nos moleste más, para ello nos adentramos en el ***call 004070A4*** para localizar el ***call*** responsable del mensaje, si existe un salto que lo evite pues perfecto y si no lo hay como en el caso anterior pues nos adentraremos en éste así hasta localizar un salto (que debe existir ya que si no estuviera desactualizada no lo mostraría), así llegamos a:

```
:004813E2 E8AD5DFCFF          call 00447194
:004813E7 C686D002000001      mov byte ptr [esi+000002D0], 01
:004813EE C686D202000001      mov byte ptr [esi+000002D2], 01
:004813F5 C686D302000001      mov byte ptr [esi+000002D3], 01
:004813FC C686D402000001      mov byte ptr [esi+000002D4], 01
:00481403 8BC6                mov eax, esi
:00481405 84DB                test bl, bl
:00481407 740F                je 00481418                ;salto que evita el mensaje_nag!
:00481409 E8F21EF8FF          call 00403300              ;aplicación desactualizada!
```

Ahora sí que tenemos lo que estábamos buscando, el call responsable del mensaje y un salto tipo condicional previo que nos aleja del mismo, para nuestro ***parche*** cambiaremos el salto tipo ***jmp***, pero esto lo veremos al

finalizar el tuto donde repasaremos los pasos del mismo así que de momento anotamos esta dirección para usarla después a la hora de parchear. Es el momento de ir a por lo grande ¡las limitaciones del programa!, nos adentramos en el mismo y antes de obligar a que nos muestre el mensaje de error pondremos un cebo en el SoftIce como por ejemplo un punto de ruptura muy común en los mensajes de error: '***bpx MessageBoxA'*** y '***bpx MessageBoxExA***' por si las moscas, ahora vamos a ***Formulario*** y seleccionamos cualquier fórmula que sobrepase la quinta posición de la lista para que el programa saque el mensaje de error, lo hacemos y antes de que salga por pantalla saltará el ***SICE*** y tomaremos el control en:

```
:0044E4C2 8B4024          mov eax, dword ptr [eax+24]
:0044E4C5 50              push eax

* Reference To: user32.MessageBoxA, Ord:0000h
                |
:0044E4C6 E8D98BFBFF      Call 004070A4
:0044E4CB 8945F8          mov dword ptr [ebp-08], eax      ;salimos aquí
:0044E4CE 33C0            xor eax, eax
:0044E4D0 5A              pop edx
```

como no hay ningún salto previo pulsamos ***F12*** para retornar al punto desde donde se llegó a esta zona que pinta el mensaje y caemos directamente en:

```
:004A8C68 A124AA4B00      mov eax, dword ptr [004BAA24]
:004A8C6D 8B00            mov eax, dword ptr [eax]
:004A8C6F 59              pop ecx
:004A8C70 E87357FAFF      call 0044E3E8
:004A8C75 8B4DFC          mov ecx, dword ptr [ebp-04]
:004A8C78 B201            mov dl, 01
```

si miramos un poco hacia arriba con los cursores en este caso sí encontramos un salto que evite esta zona del error

```
:004A8BF7 83B86003000000  cmp dword ptr [eax+00000360], 00000000
:004A8BFE 0F84BD020000    je 004A8EC1
:004A8C04 8B45FC          mov eax, dword ptr [ebp-04]      ;este nos aleja del call malo!
:004A8C07 80B87D03000000  cmp byte ptr [eax+0000037D], 00
:004A8C0E 0F84BB000000    je 004A8CCF                      ;este nos aleja del call malo!
:004A8C14 6A30            push 00000030
```

tenemos no uno sino dos saltos que nos envían lejos del error, como siempre deberemos ser lo menos restrictivos y forzar el segundo salto que no nos envía ten lejos como el primero, como hicimos antes nos anotamos la dirección para cuando hagamos el parche y seguimos con nuestro trabajo ;)

Bueno sabemos que existen más restricciones, pues el salto anterior sólo nos sirve para el campo ***Formulario***, así como tenemos activo el ***breakpoint*** en ***SoftIce*** con el que trabajamos anteriormente, así pues usaremos el mismo método: forzaremos la aparición del mensaje para ver quien lo lanza y evitarlo. Para el caso de la opción ***Conversor*** tenemos que el mensaje lo saca en:

```
:004AB9B6 8B00            mov eax, dword ptr [eax]
:004AB9B8 59              pop ecx
:004AB9B9 E82A2AFAFF      call 0044E3E8                    ;call responsable del mensaje de error.
:004AB9BE 8B4DFC          mov ecx, dword ptr [ebp-04]
:004AB9C1 B201            mov dl, 01
```

Si nos fijamos el call apunta a la misma dirección que es donde está la función ***API MessageBoxA*** que estamos rastreando, la cosa está clara tenemos una función que es llamada desde varios puntos del programa, si miramos poco más arriba como en el caso anterior veremos un salto que nos aleja de esta zona (en este caso también son dos igual que antes) pero a la misma dirección:

```
:004AB939 80B8FC02000000  cmp byte ptr [eax+000002FC], 00
:004AB940 0F84E0000000    je 004ABA26                      ;este nos aleja del call malo!
```

```
:004AB946 8B45FC          mov eax, dword ptr [ebp-04]
:004AB949 8B80EC020000    mov eax, dword ptr [eax+000002EC]
:004AB94F E890C3F7FF      call 00427CE4
:004AB954 83F804          cmp eax, 00000004
:004AB957 0F8EC9000000    jle 004ABA26                ;este nos aleja del call malo!
```

Así que como siempre adoptaremos la opción menos restrictiva que para este caso será tomar el segundo salto, lo forzaremos a que salte siempre, anotamos la dirección para luego y seguimos…

Para el caso de las ***magnitudes*** tenemos la misma llamada pero desde la posición ***:004AB770***, se evita desde:

```
:004AB6F0 80B8FC02000000  cmp byte ptr [eax+000002FC], 00
:004AB6F7 0F84E0000000    je 004AB7DD                 ;este nos aleja del call malo!
:004AB6FD 8B45FC          mov eax, dword ptr [ebp-04]
:004AB700 8B80DC020000    mov eax, dword ptr [eax+000002DC]
:004AB706 E8D9C5F7FF      call 00427CE4
:004AB70B 83F804          cmp eax, 00000004
:004AB70E 0F8EC9000000    jle 004AB7DD                ;este nos aleja del call malo!
:004AB714 6A30            push 00000030
```

que como ya es sabido forzaremos el segundo de estos saltos, nos apuntamos su dirección y seguimos buscando…, para el campo ***Estadística*** tenemos la misma llamada desde ***:004AD2A2*** y se evita desde:

```
:004AD224 80B84C03000000  cmp byte ptr [eax+0000034C], 00
:004AD22B 0F84D0000000    je 004AD301                 ;este nos aleja del call malo!
:004AD231 8B45FC          mov eax, dword ptr [ebp-04]
:004AD234 8B8051030000    mov eax, dword ptr [eax+00000351]
:004AD23A 0345E8          add eax, dword ptr [ebp-18]
:004AD23D 83F814          cmp eax, 00000014
:004AD240 0F8EBB000000    jle 004AD301                ;este nos aleja del call malo!
:004AD246 6A30            push 00000030
```

Como ya viene siendo habitual tenemos dos saltos que evitan la aparición del mensaje de error, nos apuntamos la dirección del segundo de ellos y continuamos, yo he mirado y no he encontrado ninguna limitación con lo que es posible que no la tenga o bien que no la haya localizado, en principio con los cambios anotados tendremos bastante, si por lo que fuera a alguien le apareciera otro mensaje similar sólo debería rastrear la llamada a ***:0044E3E8*** y ver desde donde se puede evitar.

Llegó el momento más emocionante, es la hora de aplicar el parche a nuestra cobaya, abrimos nuestro editor hexa preferido (***UltraEdit32*** en mi caso) y cargamos el ejecutable ***CalcMAT.exe*** con el que trabajaremos, como tenemos las direcciones anotadas en un papel sólo deberemos ir aplicando los cambios y salvar los cambios al finalizar, nada más.

Aun con estos cambios seguimos teniendo el problema de ver la nag cada vez que corramos el programa tanto al entrar como al salir, pero todo tiene solución, vamos a tirar de recursos para ver que podemos hacer, si pensamos un poco tenemos que si obtenemos el ***manejador*** (***handle***) de la ventana podremos hacer bastantes cosas, para ello y con la nag en pantalla nos vamos al SoftIce con ***ctrl.+D*** y ponemos el comando ***hwnd*** para que nos muestre los manejadores de las aplicaciones en curso, se desplegará una lista de los mismos en la que deberemos buscar la de la nag del programa ***CalcMAT***, encontraremos lo siguiente:

```
CclcMAT    frmNAG    #####      ;donde los # corresponden con el valor del handle de la ventana, notar
                                ;que cambiará en cada ejecución del programa.
```

Cuando lo tengamos haremos un seguimiento de la ventana cuando al cerrarla, para ello escribimos en ***SICE***:

```
Bmsg    ####    wm_destroy      ;recordad los # serán el valor del manejador!
```

Con esto le estamos diciendo al SoftIce que rompa en el momento de cerrar la ventana que no es más ni menos que la nag que estamos intentando eliminar, pulsamos F5 y SICE rompe justo en:

```
:00433263 50                    push eax

* Reference To: user32.DestroyWindow, Ord:0000h
                |
:00433264 E85B3BFDFF            Call 00406DC4
:00433269 85C0                  test eax, eax           ;salimos aquí!!
:0043326B 7505                  jne 00433272
```

si trazamos hacia atrás con ***F12*** para seguir la llamada no encontramos nada (al en modo de ***evaluación*** menos en mi caso) con lo que dejamos esta línea de investigación, vamos a cambiar de táctica, trazaremos desde el principio, lo hacemos para encontrar lo siguiente:

```
:004B2AE8 FF92D8000000          call dword ptr [edx+000000D8]
```

aquí tenemos la nag inicial en pantalla, si miramos un poco hacia arriba tenemos un salto que la evita en:

```
:004B2AB9 E8A2300000            call 004B5B60
:004B2ABE 807DEF00              cmp byte ptr [ebp-11], 00
:004B2AC2 7447                  je 004B2B0B             ;salto que evita la nag inicial
```

Ya hemos pillado el salto sólo queda forzarlo para que salte siempre y de esta manera evitar la nag, pero tenemos que cuando salimos la volvemos a ver, esto es una pasada no hay manera de deshacernos de ella, la elliminamos por un lado y aparece por otro, y lo que es peor todavía esta no se por dónde sale, luego ¿por dónde atacar? La respuesta está en el retardo que presenta la ventana, si os fijáis bien antes de activarse el botón de ***Continuar*** existe un retraso (***delay***) de unos pocos segundos, este será nuestro punto de ataque, localicemos la función ***API SetTimer*** de la librería ***user32.dll*** que es la encargada del retardo de la misma, hacemos una búsqueda de ésta con un punto de ruptura en la misma, asi pues entramos en ***SICE*** y ponemos ***bpx SetTimer*** justo antes de salir del programa para asegurarnos que romperá en el lugar idóneo, justo en:

```
:004556EA 50                    push eax

* Reference To: user32.SetTimer, Ord:0000h
                |
:004556EB E8CC1AFBFF            Call 004071BC
:004556F0 85C0                  test eax, eax           ;aquí salimos!!
:004556F2 7521                  jne 00455715
:004556F4 8D55FC                lea edx, dword ptr [ebp-04]
```

Pulsamos ***F12*** para retornar de la función hasta caer en:

```
:004B6AAF C78020030000DC050000 mov dword ptr [ebx+00000320], 000005DC
:004B6AB9 8B45FC                mov eax, dword ptr [ebp-04]
:004B6ABC 8B10                  mov edx, dword ptr [eax]
:004B6ABE FF92D8000000          call dword ptr [edx+000000D8]       ;aquí pinta la nag!!
:004B6AC4 33C0                  xor eax, eax
:004B6AC6 5A                    pop edx
:004B6AC7 59                    pop ecx
```

Hemos localizado el punto exacto en el que saca la nag, miramos por encima para localizar un salto que la evita en ***:004B6A8B*** que cambiamos por un ***jmp*** y salimos sin la nag ¡Al fin! Hemos solventado el problema de la ventana, pero si aplicamos los cambios vemos que el nuevo ejecutable no funciona ¿qué pasa? Fácil, tiene un ***checksum*** que realiza una comprobación de todos sus bytes y ha detectado nuestros cambios, pues no funcionará si algún cracker lo ha modificado o un virus lo ha infectado, el muy astuto ha detectado nuestra presencia! Tenemos dos posibilidades:

1.- Crear un loader que parchee en memoria nuestro exe como ***R!sc´s Process Patcher*** por ejemplo.
2.- Localizar y eliminar el cheksum.

En este tuto optaremos por la segunda opción, vamos a localizarlo y evitarlo, yo lo he hecho trazando el ejecutable original y luego el crack para encontrar las diferencias, he aquí:

```
:004B7EC8 750E                  jne 004B7ED8
```

Este salto no lo toma el original pero si el modificado, con lo que ***nopeando*** dicho salto evitamos el ***checksum***, jeje.. pero todavía hay más puesto que evitamos la nag y la citada comprobación pero.. las funciones deshabilitadas? Esas todavía siguen en pie.!! Como localizarlas no tiene ningún secreto enumeraremos los lugares donde hay que ***jumpear*** para que funcione:

Como digo encontrar el lugar donde modificar es muy sencillo si seguiste la primera parte del tut, sólo tenemos que seguirle la pista al mensaje que muestra...

Los saltos que debemos ***jumpear*** son estos:

:00481407 740F	je 00481418
:004AB957 0F8EC9000000	jle 004ABA26
:004AB70E 0F8EC9000000	jle 004AB7DD
:004AD240 0F8EBB000000	jle 004AD301
:004B7EC8 750E	jne 004B7ED8
:004B6A8B 7454	je 004B6AE1
:004B2AC2 7447	je 004B2B0B

Aplicamos los cambios con nuestro editor hexadecimal preferido (***UltraEdit32*** en mi caso) y guardamos los cambios, en este nuevo ejecutable resultante ahora sí podemos utilizar la función que más nos guste ;-)

Terminamos ya por hoy, si alguien quiere comentarme cualquier cosa puede hacerlo en:

syxe00@yahoo.es
syxe00@hotmail.com